

顺序表实现学生成绩管理系统

AI交流记录

Q:

@/work_1.c

当前项目基于C语言使用顺序表的方法建造一个学生成绩管理系统，检查其内容，完善change_data()函数，并指出其存在哪些bug及需要修改的地方

A:

一、原代码存在的 Bug 及问题

1. 语法错误

- **第14行缺少分号**： `int score_3` 后面缺少 `;`，导致编译错误。

2. change_data() 函数缺失

- **第183行**调用了 `change_data()`，但整个文件中根本没有定义该函数，导致链接错误。

3. main() 中 change_data() 调用缺少参数

- 即使定义了 `change_data()`，第183行 `change_data();` 也没有传入 `Sqlist *L` 参数。

4. 内存分配错误

- **第137行**： `Sqlist *DATA = (Sqlist*)malloc(sizeof(Sqlist)*50);` 分配了 50 个 `Sqlist` 结构体的空间，但只需要 1 个。正确应为 `malloc(sizeof(Sqlist))`。这导致内存浪费且逻辑错误。

5. 内存泄漏

- **第196行**： `free(DATA);` 只释放了 `Sqlist` 结构体本身，但没有释放其内部的 `elem` 数组（通过 `init_list` 中 `malloc` 分配的），造成内存泄漏。

6. 菜单循环逻辑问题

- 菜单只显示一次，且 `scanf` 在 `while` 循环**之前**，导致用户只能选择一次操作，无法持续交互。
- 菜单应放在循环内部，每次操作后重新显示。

7. 添加数据时的提示文字错误

- **第166-168行：**三个成绩的提示都写成了 "成绩1:"，应分别为 "成绩1:"、"成绩2:"、"成绩3:"。

8. 添加位置逻辑错误

- 用户输入位置 `pos` 后，直接传给 `insert_at(DATA, pos, add)`，但 `insert_at` 的 `index` 参数是从 0 开始的，而用户输入是从 1 开始的，应转换为 `pos - 1`。

9. scanf 参数错误

- **第161-163行：**`scanf("%s", &(add.id))` 中，数组名 `add.id` 本身就是地址，不需要取地址符 `&`。虽然在某些编译器上不会报错，但不符合规范。同理 `add.name`。

10. del_data() 空指针检查不完整

- 第70行只检查了 `L == NULL`，但未检查 `L->length == 0`，当表为空时仍会执行后续逻辑。

11. print_data() 缺少返回值

- 第132行函数结束处没有 `return` 语句，但函数声明为 `int` 类型。

12. print_data() 和 search_data() 输出格式问题

- 输出中 `score_2` 和 `score_3` 前面有多余空格（" scoore_2" 和 " score_3"），且 `score_2` 拼写为 `scoore_2`（多了一个 o）。

13. 缺少 case 1 的作用域括号

- 在 `case 1` 中声明了变量 `int pos` 和 `data add`，但在 C89/C99 标准中，`switch` 的 `case` 中声明变量需要用 `{}` 括起来，否则可能编译报错。

二、change_data() 函数实现说明

完善后的 `change_data()` 函数功能：

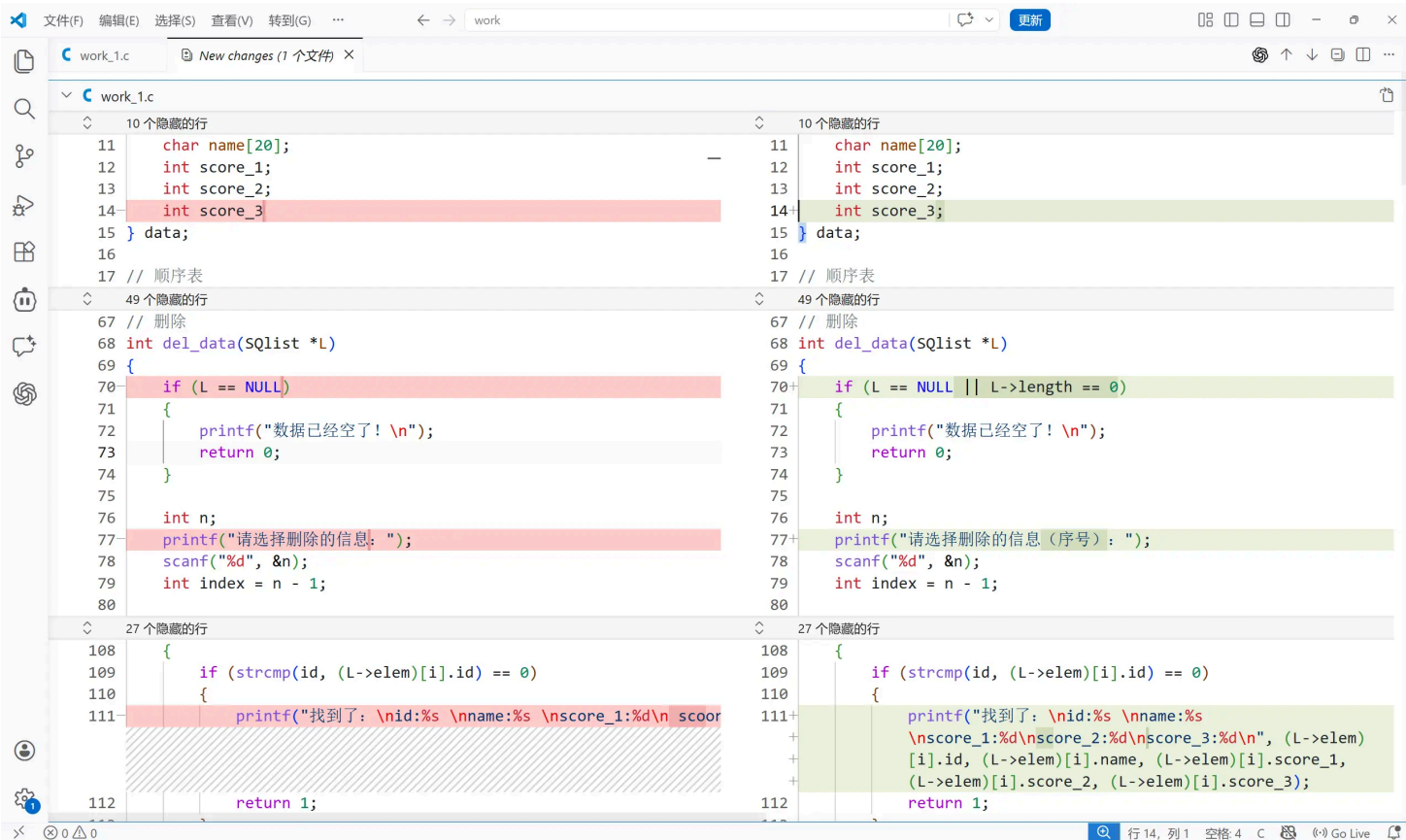
1. 检查顺序表是否为空
2. 通过学号查找要修改的学生
3. 显示当前学生信息
4. 提供子菜单让用户选择要修改的字段（姓名、成绩1、成绩2、成绩3）
5. 支持多次修改不同字段，直到用户选择返回
6. 修改后直接更新顺序表中的数据

三、其他改进

- 将菜单显示和 scanf 移入 while 循环内部，实现持续交互
- 添加了 init_list 和 malloc 的失败检查
- 完善了内存释放逻辑（先释放 elem 数组，再释放 DATA 结构体）
- 添加了操作成功/失败的提示信息

```
180         break;
181
182     case 4:
183         change_data();
184         break;
185
186     case 5:
187         print_data(DATA);
188         break;
189
190     default:
191         break;
192 }
193 printf("退出系统!\n");
194
196 free(DATA);
197 DATA = NULL;
198
199 return 0;
200 }
```

```
273         break;
274
275     case 4:
276         change_data(DATA);
277         break;
278
279     case 5:
280         print_data(DATA);
281         break;
282
283     default:
284         printf("输入错误, 请重新选择! \n");
285         break;
286 }
287 printf("退出系统!\n");
288
289 // 先释放elem数组, 再释放DATA结构体
290 if (DATA != NULL)
291 {
292     if (DATA->elem != NULL)
293     {
294         free(DATA->elem);
295         DATA->elem = NULL;
296     }
297     free(DATA);
298     DATA = NULL;
299 }
300
301 return 0;
302 }
```



项目源码

```
/******          顺序表实现学生成绩管理系统          *****/
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// 数据元素
typedef struct
{
    char id[20];
    char name[20];
    int score_1;
    int score_2;
    int score_3;
} data;

// 顺序表
typedef struct
{
    data *elem;
    int length;
    int capacity;
} SQList;

// 初始化
int init_list(SQList *L)
{
    L->elem = (data *)malloc(sizeof(data) * 10);
    if (L->elem == NULL)
        return 0; // 内存申请失败
    L->length = 0;
    L->capacity = 10;
    return 1;
}

// 尾部插入
// int push_back(SQList *L, data new_data)
// {
//     if (L->length >= L->capacity)
//     {
//         printf("数据满了! \n");
```

```

//      return 0;
//  }
//      L->elem[L->length] = new_data;
//      L->length++;
//      return 1;
// }

// 指定位置插入
int insert_at(SQlist *L, int index, data new_data)
{
    if (index < 0 || index > L->length)
        return 0; // 位置非法
    if (L->length >= L->capacity)
        return 0; // 数据满了

    // 将插入位置后面的元素从最后一个元素开始向后移，为将要插入的位置腾出空间
    for (int i = L->length; i > index; i--)
    {
        L->elem[i] = L->elem[i - 1];
    }
    L->elem[index] = new_data;
    L->length++;
    return 1;
}

// 删除
int del_data(SQlist *L)
{
    if (L == NULL || L->length == 0)
    {
        printf("数据已经空了! \n");
        return 0;
    }
    int n;
    printf("请选择删除的信息（序号）：");
    scanf("%d", &n);
    int index = n - 1;
    if (index < 0 || index > L->length - 1)
    {
        printf("选择错误! \n");
        return 0;
    }
}

```

```

    for (int i = index; i + 1 < L->length; i++)
    {
        L->elem[i] = L->elem[i + 1];
    }
    L->length--;
    printf("删除成功! \n");
    return 1;
}

// 查找信息
int search_data(SQlist *L)
{
    if (L == NULL || L->length == 0)
    {
        printf("数据为空! ");
        return 0;
    }
    char id[20];
    printf("请输入学号: \n");
    scanf("%s", id);
    for (int i = 0; i < L->length; i++)
    {
        if (strcmp(id, (L->elem)[i].id) == 0)
        {
            printf("找到了: \nid:%s \nname:%s \nscore_1:%d\nscore_2:%d\nscore_3:%d\n", (L->elem)
                return 1;
            }
        }
    }
    printf("没找到! \n");
    return 0;
}

// 修改信息
int change_data(SQlist *L)
{
    if (L == NULL || L->length == 0)
    {
        printf("数据为空! \n");
        return 0;
    }

    char id[20];
    printf("请输入要修改的学号: \n");

```

```
scanf("%s", id);
int found = -1;
for (int i = 0; i < L->length; i++)
{
    if (strcmp(id, (L->elem)[i].id) == 0)
    {
        found = i;
        break;
    }
}

if (found == -1)
{
    printf("没找到该学生! \n");
    return 0;
}

printf("当前学生信息: \n");
printf("id:%s name:%s score_1:%d score_2:%d score_3:%d\n",
        L->elem[found].id, L->elem[found].name,
        L->elem[found].score_1, L->elem[found].score_2, L->elem[found].score_3);

int choice = 0;
while (1)
{
    printf("\n请选择要修改的字段: \n");
    printf("1.姓名 2.成绩1 3.成绩2 4.成绩3 0.返回\n");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            printf("请输入新的姓名: \n");
            scanf("%s", L->elem[found].name);
            printf("修改成功! \n");
            break;
        case 2:
            printf("请输入新的成绩1: \n");
            scanf("%d", &(L->elem[found].score_1));
            printf("修改成功! \n");
            break;
        case 3:
            printf("请输入新的成绩2: \n");
```



```

        scanf("%d", &(L->elem[found].score_2));
        printf("修改成功! \n");
        break;
    case 4:
        printf("请输入新的成绩3: \n");
        scanf("%d", &(L->elem[found].score_3));
        printf("修改成功! \n");
        break;
    case 0:
        return 1;
    default:
        printf("输入错误, 请重新选择! \n");
        break;
    }
}
}

```

// 显示当前信息

```

int print_data(SQLlist *L)
{
    if (L == NULL || L->length == 0)
    {
        printf("信息为空! \n");
        return 0;
    }
    for (int i = 0; i < L->length; i++)
    {
        printf("第%d个学生: \n", i + 1);
        printf("id:%s \nname:%s \nscore_1:%d\nscore_2:%d\nscore_3:%d\n", (L->elem)[i].id, (L->e
        printf("\n");
    }
    return 1;
}

```

```

int main()
{
    // 动态创建数据表
    SQLlist *DATA = (SQLlist *)malloc(sizeof(SQLlist));
    if (DATA == NULL)
    {
        printf("内存申请失败! \n");
        return 0;
    }
}

```

```

}

// 初始化
if (init_list(DATA) == 0)
{
    printf("内存申请失败! \n");
    free(DATA);
    return 0;
}

int select = 0;
while (1)
{
    printf("\n***** 1.增添信息    2.删除信息 *****\n");
    printf("***** 3.查找信息    4.修改信息 *****\n");
    printf("***** 5.查看信息    0.退出系统 *****\n");
    printf("请选择: ");
    scanf("%d", &select);

    if (select == 0)
        break;
    switch (select)
    {
    case 1:
    {
        int pos;
        printf("添加位置 (第几个位置): \n");
        scanf("%d", &pos);
        data add;
        printf("添加数据: \n");
        printf("学号: \n");
        scanf("%s", add.id);
        printf("姓名: \n");
        scanf("%s", add.name);
        printf("成绩1:\n");
        scanf("%d", &(add.score_1));
        printf("成绩2:\n");
        scanf("%d", &(add.score_2));
        printf("成绩3:\n");
        scanf("%d", &(add.score_3));

        if (insert_at(DATA, pos - 1, add) == 0)
            printf("添加失败! \n");
    }
    }
}

```

```

        else
            printf("添加成功! \n");
        break;
    }

    case 2:
        del_data(DATA);
        break;
    case 3:
        search_data(DATA);
        break;
    case 4:
        change_data(DATA);
        break;
    case 5:
        print_data(DATA);
        break;
    default:
        printf("输入错误, 请重新选择! \n");
        break;
    }
}
printf("退出系统!\n");

// 先释放elem数组, 再释放DATA结构体
if (DATA != NULL)
{
    if (DATA->elem != NULL)
    {
        free(DATA->elem);
        DATA->elem = NULL;
    }
    free(DATA);
    DATA = NULL;
}
return 0;
}

```

链表实现学生管理系统

AI交流记录

Q:

@/work_2.c

刚项目基于C语言链表知识设计的学生成绩管理系统，检查当前代码，指出bug及需要改善的地方并给出解释

A:

`work\_2.c` 代码审查报告

一、严重 Bug（会导致程序崩溃或行为异常）

Bug 1: `scanf` 漏写取地址符号 `&`（case 3 查找功能）

****位置****: `main()` 函数中 case 3 分支，第 68 行

```c

scanf("%d", id); // 错误：漏写 & 符号

**影响**：传递给 scanf 的是 id 变量的值（非初始化状态为垃圾值），而非 id 的地址。scanf 会将该垃圾值视为地址进行写入，**必然导致段错误（Segmentation Fault）或程序崩溃**。

**修正**：改为 scanf("%d", &id);

## Bug 2: insertStudent 函数缺少返回类型

位置：第 107 行

```
insertStudent(Student *newNode)

{
```

**影响：**C 语言中未显式指定返回类型时默认返回 `int`，但该函数实际不返回任何值。**这是一个语法警告级别的错误**，严格来说不符合 C99/C11 标准，在某些编译器设置下可能导致编译失败。

**修正：**在函数名前加 `void`

```
void insertStudent(Student *newNode)
```

## Bug 3: isIdExist 函数在 case 1 中被调用时，id 变量存放的是前一次操作的残留值

**位置：**在 case 1 中，`scanf` 读取 `id` 后立即调用 `isIdExist(id)`，这段逻辑本身正确。但在 case 3 中，`scanf("%d", id)` (Bug 1) 写入错误的内存位置，可能导致 `id` 变量的值被意外修改。

**影响：**因 Bug 1 的 `scanf` 错误写入不可预测的内存区域，`id` 变量的实际值可能改变，后续所有使用 `id` 的操作结果都将不可预测。

## Bug 4: insertStudent 未检查重复学号（防御性不足）

位置：第 107-131 行

**影响：**虽然在 main 的 case 1 中已经调用 `isIdExist` 做了检查，但如果未来有其他代码绕过 main 直接调用 `insertStudent`，则可能插入重复学号。`insertStudent` 作为公共接口应自行保证数据完整性。

**修正：**在 `insertStudent` 内部添加学号重复检查。

## Bug 5: freeList 后未对全局变量 head 置 NULL 的影响

虽然代码中已经做了 `head = NULL`，但考虑以下场景：

如果 `freeList()` 被调用后，程序没有立即 `exit` 而是继续运行（例如未来重构为多次初始化/释放），由于 `head` 已被置 `NULL`，后续操作可以安全进行。目前代码中调用 `freeList` 后直接 `break` 跳出 `switch`，但外层 `while(1)` 循环仍在继续，此时 `head == NULL`，后续操作会正确处理空链表状态。然而 `case 6` 本应该退出程序，却只是 `break` 跳出 `switch` 继续循环。

这是一个逻辑 Bug：用户选择“退出”后，程序释放内存但没有退出，而是再次显示菜单。

修正：在 `case 6` 的 `freeList()` 之后应调用 `exit(0)` 或 `return 0` 退出程序。

## 二、严重性较低的 Bug / 设计缺陷

### 缺陷 1: scanf 输入字符串可能缓冲区溢出

位置：第 49 行

```
scanf("%s", name); // name 大小为 50
```

问题：`scanf("%s")` 不限制读取长度，用户输入超过 49 个字符（含 `null` 终止符）时会缓冲区溢出，破坏栈上其他变量的值。

改进：

```
scanf("%49s", name);
```

或使用 `fgets` 等更安全的输入函数。

### 缺陷 2: 菜单输入不处理非法字符

位置：`scanf("%d", &choice)`

**问题：**如果用户输入非数字字符（如 'a'），scanf 会返回 0，输入缓冲区中的 'a' 未被消费，导致无限循环输出"无效选择"。

**改进：**检查 scanf 返回值，若读取失败则清空输入缓冲区。

### 缺陷 3：全局变量 head 不利于扩展和维护

**问题：**使用全局链表头指针 head，使得程序只能管理"一个"学生链表，无法支持多班级、多数据源等场景。应封装为局部变量传递给各个函数。

**改进：**将 head 改为 main 中的局部变量，并在所有操作函数中作为参数传递。

### 缺陷 4：scanf 读取成绩后，换行符留在缓冲区

**位置：**第 52 行

**问题：**scanf("%f", &score) 读取成绩后会留一个 \n 在输入缓冲区。对于当前菜单交互没问题，但如果后续添加需要读取字符的操作（如 getchar），\n 会被意外读走。

**改进：**读取后可以增加 getchar() 消费残留换行符。

### 缺陷 5：displayAll 输出格式未对齐

**问题：**使用 \t 制表符对齐，但当学号/姓名字符串长度不同时，制表位会导致列不对齐。

**改进：**使用 %-10d %-20s %-8.2f 等格式化输出确保对齐。

### 缺陷 6：isIdExist 函数重复遍历链表

**问题：**在 insertStudent 前调用 isIdExist 会遍历一次链表，随后 insertStudent 插入时又可能再次遍历链表找到插入位置，对于长链表效率较低。

**改进：**可让 insertStudent 返回插入状态（成功/失败），在插入时一并完成重复检查，省去提前查找的开销。

# 缺陷 7：代码没有包含 <conio.h> 或使用 getch 暂停程序

**问题：**在 Windows 上运行 .exe 时，程序可能在执行完毕后立即关闭控制台窗口，用户来不及看到结果。某些操作后程序直接进入下一轮循环，用户希望暂停查看结果。

**改进：**可在关键操作后添加 system("pause");（Windows）或通用的 getch() 暂停。

## 三、总结

| 严重程度 | 问题                    | 影响        | 是否必须修复 |
|------|-----------------------|-----------|--------|
|      |                       |           |        |
| 严重   | scanf("%d", id) 漏写 &  | 查找功能必崩溃   | 必须     |
| 严重   | insertStudent 缺少返回类型  | 编译警告/错误   | 必须     |
| 严重   | case 6 退出后没有真正退出      | 无法退出程序    | 必须     |
| 中等   | scanf("%s") 缓冲区溢出     | 输入过长时崩溃   | 建议修复   |
| 中等   | scanf 非法字符导致死循环       | 输入字母后无法继续 | 建议修复   |
| 中等   | insertStudent 未自查重复学号 | 数据可能不一致   | 建议修复   |
| 建议   | 全局变量、输出格式、效率等         | 代码质量和维护性  | 锦上添花   |



# 项目源码

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

// 学生结构体

typedef struct Student

{

 int id; // 学号（唯一标识）

 char name[50]; // 姓名

 float score; // 成绩

 struct Student *next; // 指向下一个节点的指针

} Student;

// 全局链表头指针（初始为空）

Student *head = NULL;

// 函数声明

Student *createStudent(int id, const char *name, float score);

void insertStudent(Student *newNode);

void deleteStudent(int id);

Student *findStudent(int id);
```

```
void updateStudent(int id);

void displayAll();

void freeList();

int isIdExist(int id);

int main()

{

 int choice, id;

 char name[50];

 float score;

 while (1)

 {

 printf("\n===== 学生成绩管理系统 =====\n");

 printf("1. 添加学生\n");

 printf("2. 删除学生（按学号）\n");

 printf("3. 查找学生（按学号）\n");

 printf("4. 修改学生成绩（按学号）\n");

 printf("5. 显示所有学生\n");

 printf("6. 退出\n");

 printf("请选择操作： ");
```

```
scanf("%d", &choice);

switch (choice)

{

case 1:

 printf("请输入学号: ");

 scanf("%d", &id);

 if (isIdExist(id))

 {

 printf("该学号已存在, 添加失败! \n");

 break;

 }

 printf("请输入姓名: ");

 scanf("%s", name); // 简单输入, 假设不含空格

 printf("请输入成绩: ");

 scanf("%f", &score);

 Student *newNode = createStudent(id, name, score);

 insertStudent(newNode);

 printf("添加成功! \n");

 break;

case 2:
```

```
printf("请输入要删除的学号：");
```

```
scanf("%d", &id);
```

```
deleteStudent(id);
```

```
break;
```

```
case 3:
```

```
printf("请输入要查找的学号：");
```

```
scanf("%d", &id);
```

```
Student *found = findStudent(id);
```

```
if (found)
```

```
{
```

```
 printf("找到学生：学号=%d，姓名=%s，成绩=%.2f\n", found->id, found->name, found->
```

```
}
```

```
else
```

```
{
```

```
 printf("未找到该学号！\n");
```

```
}
```

```
break;
```

```
case 4:
```

```
printf("请输入要修改成绩的学号：");
```

```
scanf("%d", &id);
```

```
updateStudent(id);
```

```
break;
```

```
case 5:
```

```
displayAll();
```

```
break;
```

```
case 6:
```

```
freeList(); // 释放所有节点内存
```

```
printf("程序退出，已释放内存。\\n");
```

```
return 0;
```

```
default:
```

```
printf("无效选择，请重新输入! \\n");
```

```
}
```

```
}
```

```
}
```

```
// 创建新节点
```

```
Student *createStudent(int id, const char *name, float score)
```

```
{
```

```
Student *newNode = (Student *)malloc(sizeof(Student));

if (!newNode)

{

 printf("内存分配失败! \n");

 exit(1);

}

newNode->id = id;

strcpy(newNode->name, name);

newNode->score = score;

newNode->next = NULL;

return newNode;

}
```

// 按学号升序插入链表（无重复学号）

```
void insertStudent(Student *newNode)

{

 // 如果链表为空，直接作为头节点

 if (head == NULL)

 {

 head = newNode;

 return;

 }

}
```

```
}
```

```
// 如果新学号小于头节点学号，插入到头部
```

```
if (newNode->id < head->id)
```

```
{
```

```
 newNode->next = head;
```

```
 head = newNode;
```

```
 return;
```

```
}
```

```
// 遍历找到合适位置（保持升序）
```

```
Student *cur = head;
```

```
while (cur->next != NULL && cur->next->id < newNode->id)
```

```
{
```

```
 cur = cur->next;
```

```
}
```

```
// 插入到 cur 之后
```

```
newNode->next = cur->next;
```

```
cur->next = newNode;
```

```
}
```

```
// 删除指定学号的学生
```

```
void deleteStudent(int id)

{

 if (head == NULL)

 {

 printf("链表为空，无法删除！ \n");

 return;

 }

 Student *cur = head;

 Student *prev = NULL;

 // 查找要删除的节点

 while (cur != NULL && cur->id != id)

 {

 prev = cur;

 cur = cur->next;

 }

 if (cur == NULL)

 {

 printf("未找到学号 %d，删除失败！ \n", id);
```



```
 return;

 }

 // 删除节点

 if (prev == NULL)

 {

 // 删除头节点

 head = cur->next;

 }

 else

 {

 prev->next = cur->next;

 }

 free(cur);

 printf("学号 %d 删除成功! \n", id);

}
```

// 按学号查找

```
Student *findStudent(int id)

{

 Student *cur = head;

 while (cur != NULL)
```

```
{

 if (cur->id == id)

 return cur;

 cur = cur->next;

}

return NULL;

}
```

// 修改学生成绩

```
void updateStudent(int id)

{

 Student *stu = findStudent(id);

 if (stu == NULL)

 {

 printf("未找到学号 %d, 修改失败! \n", id);

 return;

 }

 printf("当前成绩为 %.2f, 请输入新成绩: ", stu->score);

 float newScore;

 scanf("%f", &newScore);

 stu->score = newScore;
```

```
printf("成绩修改成功! \n");

}

// 显示所有学生信息

void displayAll()

{

 if (head == NULL)

 {

 printf("链表为空, 暂无学生信息. \n");

 return;

 }

 printf("\n所有学生信息 (按学号升序): \n");

 printf("学号\t姓名\t成绩\n");

 Student *cur = head;

 while (cur != NULL)

 {

 printf("%d\t%s\t%.2f\n", cur->id, cur->name, cur->score);

 cur = cur->next;

 }

}
```

```
// 检查学号是否已存在
```

```
int isIdExist(int id)

{

 return findStudent(id) != NULL;

}
```

// 释放整个链表

```
void freelist()

{

 Student *cur = head;

 while (cur != NULL)

 {

 Student *tmp = cur;

 cur = cur->next;

 free(tmp);

 }

 head = NULL;

}
```